



CONECTAR

 SESIÓN 12 · UNIDAD III · WEB 3.0

CONECTAR LA PANTALLA A LA CADENA

Hasta hoy todo pasaba en la terminal y el explorador. Hoy, una interfaz que una persona no técnica puede usar, y que no le miente.

Blockchain y Web 3.0 · Ingeniería de Sistemas
Universidad de San Buenaventura Medellín

CÓMO VAMOS A USAR LAS TRES HORAS

La sesión más de «producto» del curso. El error que se combate: creer que una transacción es instantánea y siempre sale bien.

A

ARQUITECTURA, ETHERS Y UX

Qué está descentralizado, RPC, Provider/Signer/Contract, eventos y los estados de error.

~65 min

B

LABORATORIO 12 · EN PAREJAS

La dApp de la FichaUSB: conectar, leer saldo, transferir y provocar los 4 errores.

~85 min

C

PROYECTO · HACIA EL AVANCE 1

Qué lee y qué escribe la interfaz del proyecto; React y wagmi.

~20 min



AL SALIR DE AQUÍ

Construir una interfaz que lea y escriba en contratos, gestionando la billetera y los estados de una transacción.

LAS PREGUNTAS QUE LA SESIÓN RESPONDE

- 01** En una dApp, ¿qué parte está de verdad descentralizada y cuál no?
- 02** ¿Qué diferencia una lectura gratis de una transacción que cuesta gas?
- 03** ¿Cómo se enteraría la interfaz de que algo cambió en la cadena?
- 04** ¿Cuáles son los cuatro errores que todo principiante ignora, y cómo se detecta cada uno?

RA6 · Integrar una interfaz Web 3.0 con contratos, gestionando wallets, eventos y estados de transacción.



SIGLAS Y TÉRMINOS DE HOY

DAPP*aplicación descentralizada*

Interfaz + contratos. La interfaz suele ser una web normal; lo descentralizado son los contratos.

PROVIDER*proveedor (ethers)*

Objeto que LEE la cadena: saldos, estado, eventos. No firma, no cuesta gas.

ABI*Application Binary Interface*

La lista de funciones y eventos del contrato que la interfaz necesita para llamarlo.

WAGMI · VIEM*librerías de React y JS*

El camino profesional: ganchos listos para conexión, lecturas y escrituras.

RPC*Remote Procedure Call*

El nodo al que la interfaz le hace preguntas: propio, de MetaMask o de Alchemy/Infura.

SIGNER*firmante (ethers)*

Objeto que ESCRIBE: pide a la billetera que firme una transacción. Cuesta gas.

EIP-1193*API del proveedor*

El estándar de window.ethereum: request(), eventos y códigos de error como 4001.

RECIBO*transaction receipt*

Lo que existe cuando la transacción se minó: bloque, gas usado y status (1 ok, 0 revertida).



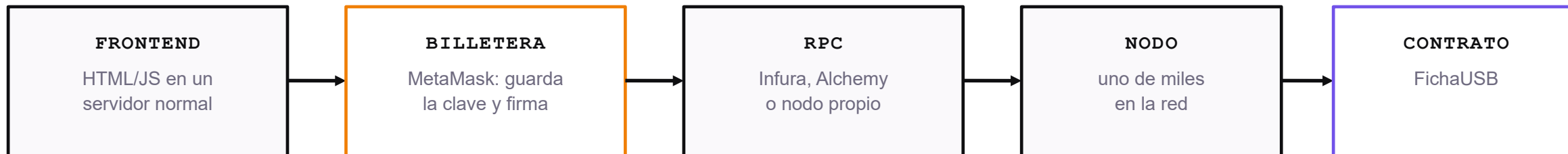
APROXIMADAMENTE 65 MINUTOS

ARQUITECTURA, ETHERS Y UX

Una dApp es, casi siempre, una web normal que además sabe hablar con una cadena. Lo interesante está en ese «además».



LAS CINCO PIEZAS DE UNA DAPP



Ejemplo trabajado: Ana pulsa «Enviar 2,5 FUSB». El frontend arma la llamada con el ABI; MetaMask le muestra qué va a firmar y firma con SU clave (el frontend nunca la ve); el RPC recibe la transacción firmada y la pasa a la red; un nodo la incluye en un bloque; el contrato mueve los saldos y emite Transfer.



EL ERROR DE CONCEPTO TÍPICO

«La página tiene acceso a mi billetera.» No: la página PIDE, la billetera decide. La clave privada nunca sale de MetaMask. Por eso la regla del curso es leer lo que la billetera muestra antes de firmar: es la única pieza que trabaja para ustedes.



LA RESPUESTA HONESTA: MENOS DE LO QUE PARECE

PIEZA	QUIÉN LA CONTROLA	SI CAE O MIENTE...
Frontend	Quien paga el hosting y el dominio.	La dApp desaparece o muestra otra cosa; el contrato sigue vivo.
RPC	Una empresa (o ustedes, si tienen nodo).	No se puede leer ni enviar; o responde datos falsos a quien no verifica.
Billetera	El usuario (software de un tercero).	Sin billetera no hay firma: solo lectura.
Contrato	Nadie: sus reglas están en la cadena.	No cae mientras la red exista. Es lo único descentralizado de verdad.



CÓMO DECIRLO CON MADUREZ

El VALOR y las REGLAS están descentralizados en el contrato; el ACCESO, muchas veces no. Se puede mejorar (frontend en IPFS, varios RPC, nodo propio), pero en el README del proyecto hay que decir con honestidad cuál es cuál.



NODO PROPIO O SERVICIO DE INFRAESTRUCTURA

NODO PROPIO

Máxima independencia: nadie les filtra ni les limita. Pero un nodo completo de Ethereum pide cientos de GB de disco rápido, sincronizar tarda horas o días, y hay que mantenerlo.

INFURA, ALCHEMY Y SIMILARES

Una URL con una clave de API y listo. Plan gratuito con límite de consultas por segundo y por mes. A cambio, dependen de una empresa que ve todas sus consultas.

SITUACIÓN	DE DÓNDE LEE LA DAPP DEL LABORATORIO
El usuario conectó MetaMask	Del RPC que MetaMask tenga configurado para esa red. La dApp no necesita clave propia.
Sin billetera (solo lectura)	De un RPC propio del frontend: su clave queda VISIBLE en el código de la página.

Una clave de API en un frontend es pública por definición: restrínjanla por dominio en el panel del proveedor. Las de Hardhat van al keystore; esta no es secreta, solo acotada.



EL RPC TAMBIÉN DICE «NO»

Todo RPC gratuito limita cuántas consultas acepta. Una interfaz mal hecha lo agota sola: por ejemplo, preguntar el saldo cada segundo desde cada pestaña abierta.

MALA PRÁCTICA	MEJOR
Preguntar el saldo en un bucle cada segundo.	Leer una vez y escuchar el evento Transfer (A.13).
Pedir todos los eventos desde el bloque 0.	Pedir un rango acotado o usar un indexador (Sesión 13).
Reintentar sin pausa cuando algo falla.	Reintentar con espera creciente y avisar al usuario.

EL SÍNTOMA

Errores 429 («too many requests») o respuestas lentas que parecen «la red está caída». En la Sesión 11 lo vimos con las puertas IPFS públicas: el mismo fenómeno, en otra capa. Es un límite comercial, no una falla de la cadena.



¿QUÉ DEJA DE FUNCIONAR?

El proveedor RPC que usa MetaMask cae durante una hora. ¿Qué le pasa a la FichaUSB, a los saldos y a su dApp? ¿Y si lo que cae es el hosting del frontend?

- Un minuto en pareja. Separen: el contrato, los datos, el acceso.



RESPUESTA

Caído el RPC: el contrato y los saldos siguen intactos en miles de nodos; la dApp no puede leer ni enviar, a menos que el usuario cambie de RPC en MetaMask. Caído el hosting: la página no carga, pero cualquiera puede interactuar con el contrato desde el explorador («Write contract») o desde otro frontend. Nada se pierde; se pierde el ACCESO cómodo.

PROVIDER, SIGNER, CONTRACT

MATERIAL/DAPP-FICHA/INDEX.HTML (RESUMIDO)

```
// PROVIDER: habla con el nodo a través de la billetera. Lee gratis.
const provider = new ethers.BrowserProvider(window.ethereum);
await provider.send("eth_requestAccounts", []); // MetaMask pide permiso

// SIGNER: la cuenta que firma. Escribir cuesta gas.
const signer = await provider.getSigner();

// CONTRACT: dirección + ABI + quién llama.
const ficha = new ethers.Contract(DIRECCION_FICHA, ABI, signer);

const bruto = await ficha.balanceOf(cuenta); // lectura
const tx     = await ficha.transfer(dest, cantidad); // escritura
```

PIEZA	PARA QUÉ	¿FIRMA?
Provider	Leer: saldos, estado, eventos, recibos.	No
Signer	Firmar y enviar transacciones.	Sí
Contract	Traducir llamadas de JS a la cadena usando el ABI.	Según la función



LO ÚNICO QUE LA INTERFAZ NECESITA DEL CONTRATO

EL ABI COMPLETO DE LA DAPP: CINCO LÍNEAS

```
const ABI = [  
  "function symbol() view returns (string)",  
  "function decimals() view returns (uint8)",  
  "function balanceOf(address) view returns (uint256)",  
  "function transfer(address to, uint256 amount) returns (bool)",  
  "event Transfer(address indexed from, address indexed to, uint256 value)",  
];
```

- Formato «legible» de ethers: se copia la firma de la función tal como en Solidity.
- Solo lo que la interfaz usa. El ABI completo está en artifacts/ tras compilar con Hardhat.
- Con él, ethers calcula el selector de 4 bytes (Sesión 5) y codifica los argumentos.
- No es secreto: el contrato verificado lo publica en el explorador.

Error típico: una coma de menos o un tipo mal escrito (uint en vez de uint256) y ethers lanza error al crear el Contract.



LA DISTINCIÓN QUE DEFINE EL COSTO

LECTURA · ETH_CALL

`ficha.balanceOf(ana)`

El nodo ejecuta la función en su copia local y responde. No hay transacción, no hay firma, no hay gas para el usuario, no queda rastro en la cadena. Instantánea.

ESCRITURA · TRANSACCIÓN

`ficha.transfer(dest, monto)`

La billetera pide firmar; la transacción viaja a la red; espera turno en un bloque; cuesta gas; puede fallar. Devuelve una tx cuyo recibo hay que esperar.

OPERACIÓN SOBRE FICHAUSB (MEDIDA EN HARDHAT)	GAS
<code>transfer a una cuenta que tenía saldo 0</code>	51 692
<code>transfer a una cuenta que ya tenía saldo</code>	34 592
<code>balanceOf llamada como lectura</code>	0 para el usuario (el nodo la ejecuta gratis)

¿Por qué la primera transferencia cuesta más? Escribir una celda de almacenamiento que estaba en cero es la operación más cara de la EVM (Sesión 5).



NO HAY DECIMALES EN LA CADENA

EJECUTADO CON ETHERS 6

```
decimales = 18
ethers.parseUnits("2.5", 18)      // -> 25000000000000000000n    (lo que viaja)
ethers.formatUnits(10000000000000000000n, 18)  // -> "1000.0"    (lo que se muestra)

Number(10000000000000000000000n)  // 1e+21 -> pierde precisión: NO
```

REGLA DE ORO

Todo lo que va o viene de la cadena es un entero BigInt (sufijo n).
Se convierte a texto legible solo para mostrar, y de texto a BigInt solo al enviar.

EL ERROR DE CONCEPTO TÍPICO

Usar Number o parseFloat con montos: 0,1 + 0,2 no da 0,3 en JavaScript, y los enteros grandes pierden dígitos. Con dinero, eso es un error de contabilidad.



«ENVIADA» NO ES «HECHA»

```
const tx = await ficha.transfer(dest, cantidad);  
// ya hay HASH: la transacción fue firmada y enviada. NO se ha transferido nada.  
  
const recibo = await tx.wait();  
// ahora sí: está en un bloque. recibo.status 1 = éxito.  
// en ethers v6, si status es 0 (revertida), wait() LANZA un error CALL_EXCEPTION.
```

01**PRIMER AWAIT**

Espera a que el usuario firme en MetaMask y a que el RPC acepte la transacción. Segundos.

02**ENTRE LOS DOS**

La transacción está pendiente en el mempool. Puede tardar, reemplazarse o no minarse nunca.

03**SEGUNDO AWAIT**

Espera a que se mine. En Sepolia, unos 12 s por bloque; en la red local, al instante.



UNA CONFIRMACIÓN NO ES FINALIDAD

tx.wait() espera por defecto UNA confirmación: el bloque que la incluye. Recuerden la Sesión 4: ese bloque todavía puede reorganizarse.

QUÉ PASA EN LA INTERFAZ	CUÁNTO ESPERAR
Mostrar un saldo actualizado, un «me gusta»	1 confirmación: tx.wait().
Liberar algo de valor (un producto, un certificado)	Varias: tx.wait(n), o esperar la finalidad.
Finalidad económica en Ethereum	Unos 13 minutos (dos épocas, Sesión 4).



DECISIÓN DE NEGOCIO, NO TÉCNICA

Cuántas confirmaciones esperar depende de cuánto se pierde si la transacción desaparece. Para el laboratorio, una basta. Para un sistema que entrega algo de valor, hay que escribir el número en el diseño y justificarlo, igual que en la Sesión 4.



¿QUÉ MUESTRA LA PANTALLA, Y CUÁNDO?

CÓDIGO DE UN ESTUDIANTE

```
estado("Confirma en la billetera...");
const tx = await ficha.transfer(dest, ethers.parseUnits(monto, 18));
estado("¡Listo! Transferencia hecha."); // (1)
const recibo = await tx.wait();
refrescarSaldo();
```

- ¿Qué está mal en la línea (1)? ¿Qué debería decir ahí?
- Si el usuario escribe 2,5 en el monto, ¿qué número exacto viaja a la cadena?
- ¿Qué pasa con este código si la transacción se revierte?

RESPUESTAS

(1) Dice «listo» cuando apenas se ENVIÓ: debe decir «enviada, esperando confirmación» y el «listo» va después de wait(). Viaja 2500000000000000000n. Si se revierte, wait() lanza CALL_EXCEPTION y, sin try/catch, la pantalla se queda en «¡Listo!»: una mentira.



QUE LA INTERFAZ SE ACTUALICE SOLA

DAPP-FICHA/INDEX.HTML

```
function suscribirEventos() {  
  contrato.on("Transfer", (from, to) => {  
    if (from === cuenta || to === cuenta) refrescarSaldo(); // solo si me toca  
  });  
}
```

SIN EVENTOS

La interfaz pregunta el saldo cada pocos segundos, cambie o no.
Gasta cuota del RPC y siempre va un poco atrasada.

CON EVENTOS

La interfaz no pregunta hasta que el contrato emite Transfer. Si un
compañero les transfiere, el saldo cambia solo, sin recargar.



LO QUE ENCONTRAMOS AL PROBAR LA DAPP DE REFERENCIA

suscribirEventos() estaba definida pero nunca se llamaba: el saldo no se refrescaba solo. Se corrigió y se probó.

LOS CUATRO ESTADOS QUE TODOS OLVIDAN

ESTADO	QUÉ VE EL USUARIO SI NO SE MANEJA	QUÉ DEBE MOSTRAR LA INTERFAZ
1 · Red equivocada	Un error críptico, o una transacción en otra red.	«Estás en la red equivocada» y el botón Enviar bloqueado.
2 · Sin billetera	La página se rompe: window.ethereum no existe.	«Instala MetaMask» y el botón Conectar desactivado.
3 · Usuario rechaza	La app queda «cargando» para siempre.	«Rechazaste la transacción» y volver al estado normal.
4 · Revertida	La app dice «éxito» cuando falló.	«Se revirtió», con el motivo y si se pagó gas o no.

Y dos estados que no son errores pero se olvidan igual: PENDIENTE (enviada, sin minar) y MINADA (con su bloque).

DEL ESTADO AL CÓDIGO

ESTADO	SEÑAL EN EL CÓDIGO (ETHERS V6 + METAMASK)	CUÁNDO APARECE
Sin billetera	<code>typeof window.ethereum === "undefined"</code>	Al cargar la página.
Red equivocada	<code>(await provider.getNetwork()).chainId !== 11155111n</code>	Al conectar, y en el evento <code>chainChanged</code> .
Rechazo	<code>e.code === "ACTION_REJECTED"</code> (4001 en EIP-1193)	Al esperar la firma.
Revertida al simular	<code>e.code === "CALL_EXCEPTION"</code> , sin <code>e.receipt</code>	Antes de firmar: <code>estimateGas</code> falla. No gasta gas.
Revertida minada	<code>e.code === "CALL_EXCEPTION"</code> , con <code>e.receipt</code>	En <code>tx.wait()</code> . El gas Sí se pagó.

Verificado en la preparación: transferir más que el saldo falla al simular con `CALL_EXCEPTION` y el mensaje «execution reverted (unknown custom error)», sin abrir MetaMask.

DE «ENVIAR» A «CONFIRMADA»

- | | | |
|----|----------------|---|
| 01 | VALIDAR | Dirección con <code>ethers.isAddress</code> y monto > 0 ANTES de tocar la billetera. Si fallan, avisar y parar. |
| 02 | PEDIR FIRMA | «Confirma en la billetera...» (amarillo). Aquí puede ocurrir el rechazo (3) o la reversión al simular (4). |
| 03 | ENVIADA | Ya hay hash: «Esperando confirmación... 0x1a2b...» (amarillo). Todavía NO decir listo. |
| 04 | ESPERAR RECIBO | <code>tx.wait()</code> . Si lanza <code>CALL_EXCEPTION</code> con recibo: revertida minada (4). |
| 05 | CONFIRMADA | «¡Listo! Confirmada en el bloque 11» (verde). Refrescar saldo; opcional: enlace al explorador. |

Cada paso tiene su mensaje. El usuario nunca debe mirar una pantalla que no dice qué está pasando.



CAMBIAR DE RED, Y VARIAS BILLETERAS

OFRECER EL CAMBIO DE RED

En vez de solo decir «red equivocada», la interfaz puede pedirle a MetaMask que cambie:

wallet_switchEthereumChain con chainId 0xaa36a7 (Sepolia).

Si la billetera no conoce esa red, responde el código 4902 y hay que ofrecer agregarla.

VARIAS BILLETERAS: EIP-6963

Con dos extensiones instaladas, ambas pelean por window.ethereum y «gana la última en cargar». EIP-6963 (estado Final) lo resuelve: cada billetera se anuncia con un evento y la dApp muestra la lista para que el usuario elija.

[-] PARA EL PROYECTO, NO PARA HOY

La dApp del laboratorio usa window.ethereum a secas: basta para aprender. wagmi y RainbowKit ya implementan las dos mejoras.

ETHERS A MANO, O REACT + WAGMI + RAINBOWKIT

WAGMI · LOS MISMOS CUATRO PASOS, COMO GANCHOS DE REACT

```
const { address, isConnected } = useAccount();
const { data: saldo } = useReadContract({ address: FICHA, abi, functionName: "balanceOf", args: [address] });
const { writeContract, isPending } = useWriteContract();
const { isSuccess } = useWaitForTransactionReceipt({ hash });
```

	ETHERS A MANO (LABORATORIO)	REACT + WAGMI + RAINBOWKIT (PROYECTO)
Se ve cada pieza	Sí: ideal para entender.	No: la magia la hacen los ganchos.
Herramientas	Un HTML y un navegador.	Node, npm y un empaquetador (p. ej. Vite).
Billeteras y red	A mano.	Incluido.

 VERSIONES, VERIFICADAS EN SEPTIEMBRE DE 2026

wagmi va en la 3.x, pero RainbowKit 2.2 exige wagmi 2.x. Si usan RainbowKit, fijen wagmi 2 (versiones congeladas).



DIAGNOSTIQUEN LA PANTALLA

LO QUE REPORTA UN USUARIO	¿QUÉ ESTADO ES?	¿QUÉ DEBIÓ MOSTRAR LA DAPP?
«Le di Enviar, cerré el MetaMask y la página quedó cargando.»	?	?
«Decía listo, pero el saldo de mi amigo no cambió.»	?	?
«Abrí el link desde el celular y no pasó nada al tocar Conectar.»	?	?



RESPUESTAS

1) Rechazo (3): «Rechazaste la transacción» y volver a habilitar el botón. 2) Dijo «listo» sin esperar el recibo: o seguía pendiente, o se revirtió (4); debía decir «esperando confirmación» y luego el resultado real. 3) Sin billetera (2): el navegador del celular no tiene window.ethereum; debía decir «instala una billetera o abre esto desde su navegador interno».



SEIS IDEAS SOBRE DAPPS

01

Lo descentralizado es el contrato; el frontend y el RPC muchas veces no. Hay que decirlo.

02

La página pide; la billetera decide y firma. La clave nunca sale de la billetera.

03

Provider lee gratis (`eth_call`); Signer escribe con una transacción que cuesta gas.

04

Todo monto es `BigInt`: `parseUnits` para enviar, `formatUnits` para mostrar.

05

`await transfer()` significa «enviada»; «hecha» lo dice `tx.wait()`. Una confirmación no es finalidad.

06

La calidad está en los cuatro errores y en los estados pendiente y minada.

B



APROXIMADAMENTE 85 MINUTOS · EN PAREJAS

LABORATORIO 12

Una interfaz que conecta la billetera, muestra el saldo de la FichaUSB, transfiere y maneja los cuatro errores. Sin compilar nada.

UN SOLO ARCHIVO, OCHO TODO

laboratorio-12/andamiaje/index.html es la dApp de referencia con las funciones clave vacías. Mientras un TODO falte, la caja «Estado» dice cuál: no falla en silencio.

TODO	QUÉ COMPLETAN	LÁMINA QUE LO EXPLICA
1	Detectar que no hay billetera (estado 2).	A.14 · A.15
2 · 3	Conectar: provider, signer, cuenta; detectar red equivocada (estado 1).	A.6 · A.15
4 · 5	Crear el Contract; leer symbol, decimals y balanceOf; formatUnits.	A.7 · A.9
6	transfer con parseUnits, mensaje de «enviada» y tx.wait().	A.10 · A.16
7	Clasificar el error: rechazo (3), revertida (4), sin gas.	A.15
8	Escuchar Transfer y refrescar el saldo solo.	A.13



DOBLE CLIC NO SIRVE: HACE FALTA UN SERVIDOR

POR QUÉ FILE:// FALLA

Chrome no deja que las extensiones actúen en páginas file:// salvo que se active un permiso por extensión, y MetaMask da permisos por SITIO: una página file:// no tiene un origen normal. Resultado típico: «no se detecta billetera».

LA SOLUCIÓN: UN SERVIDOR LOCAL

Desde la carpeta andamiaje:

```
npx serve .  
→ abrir http://localhost:3000
```

o, si no hay Node:

```
python -m http.server 8000  
→ abrir http://localhost:8000
```



NO ACTIVEN «PERMITIR ACCESO A URL DE ARCHIVO»

Funcionaría a veces, pero da a la extensión acceso a sus archivos locales. La práctica correcta es servir por http://localhost, igual que en producción se sirve por https.



RED LOCAL (SALA) O SEPOLIA (CASA)

OPCIÓN A · RED LOCAL DE HARDHAT

```
# terminal 1 · se deja abierta
npx hardhat node

# terminal 2 · despliega FichaUSB, les da 1000 FUSB y 10 ETH locales
$env:DESTINO="0xSU_DIRECCION_DE_METAMASK"
npx hardhat run scripts/s12/preparar-local.js --network localhost
```

- En MetaMask: agregar red RPC `http://127.0.0.1:8545`, chainId 31337, moneda ETH.
- Ninguna clave privada se importa: el script les manda fondos a SU cuenta.
- Opción B: la FichaUSB que desplegaron en la S10, en Sepolia.

SALIDA ESPERADA (LA DIRECCIÓN PUEDE VARIAR)

```
DIRECCION_FICHA : 0x9fE46736679d2D9a65F0992F2272dE9f3c7fa6e0
chainId         : 31337
FUSB            : 1000.0
ETH (local)    : 10.0
```

Si reinician el nodo, la cadena se borra: se vuelve a correr el script y se limpia la actividad de MetaMask.



CONECTAR Y LEER, DESPUÉS ESCRIBIR

- | | | |
|-----------|------------|--|
| 01 | TODO 1 A 3 | Conectar. Se espera: cuenta 0x..., red «unknown (31337)» o «sepolia (11155111)». |
| 02 | TODO 4 Y 5 | Leer. Se espera: «Saldo FUSB 1000.0 FUSB» y el estado verde «Conectado». |
| 03 | TODO 6 | Transferir 2,5 a un compañero. Amarillo «esperando confirmación», luego verde con el bloque. |
| 04 | TODO 7 | Errores clasificados: cada uno con su mensaje rojo propio (lámina siguiente). |
| 05 | TODO 8 | Que un compañero les transfiera: el saldo cambia SIN recargar la página. |

Después de cada TODO: guardar, recargar el navegador (F5) y probar. Nada se compila.

PROVOCAR LOS CUATRO ERRORES A PROPÓSITO

ESTADO	CÓMO PROVOCARLO	MENSAJE ESPERADO
1 · Red equivocada	En MetaMask, cambiar a otra red (p. ej. Sepolia si esperan 31337) y reconectar.	«Red equivocada...» y Enviar bloqueado.
2 · Sin billetera	Abrir la página en otro navegador o ventana sin MetaMask.	«No se detecta una billetera...»
3 · Rechazo	Pulsar Enviar y, en MetaMask, «Rechazar».	«Rechazaste la transacción...»
4 · Revertida	Transferir más FUSB de los que tienen.	«Se revirtió al simularla... no se gastó gas».

EVIDENCIA

Una captura por estado, más una de pendiente y una de confirmada. Seis capturas: es lo que demuestra que se entendió la UX.

ERRORES COMUNES DEL LABORATORIO 12

SÍNTOMA	CAUSA Y ARREGLO
«No se detecta una billetera» teniendo MetaMask	Abrieron el archivo con doble clic (file://). Sírvanlo con npx serve o python -m http.server.
«could not decode result data» al leer	DIRECCION_FICHA no tiene contrato en ESA red: dirección o red equivocada, o reiniciaron el nodo.
MetaMask: «nonce too high» en red local	Reiniciaron el nodo: en MetaMask, borrar los datos de actividad de la cuenta (configuración avanzada).
Red siempre «equivocada»	CHAIN_ID_SEPOLIA no coincide: 31337n en local; comparen BigInt con BigInt (con la n).
El saldo no se actualiza solo	Falta llamar suscribirEventos() después de conectar (TODO 4 y 8).
ethers is not defined	Sin internet no carga ethers desde jsDelivr. Recarguen con conexión.



EVIDENCIA DEL LABORATORIO 12

EVIDENCIA	DETALLE	PESO
Repositorio	index.html con los 8 TODO completos, en el repo del equipo.	25 %
Transferencia real	Captura de «confirmada» con su hash (en Sepolia, enlace al explorador).	20 %
Los cuatro errores + 2 estados	Seis capturas, cada estado provocado a propósito.	35 %
Eventos	Saldo actualizado sin recargar tras una transferencia de un compañero.	10 %
Reflexión	¿Qué parte de su dApp NO está descentralizada? (tabla A.2).	10 %

DÓNDE ESTÁ TODO

Guía exageradamente detallada: laboratorio-12/guia/guia-laboratorio-12.pdf. Terminó cuando las seis capturas existen y el saldo cambia solo. Sin hash de transacción, no se califica.



C

 APROXIMADAMENTE 20 MINUTOS

EL PROYECTO

Conectar la interfaz propia a los contratos propios. El primer momento en que el proyecto se ve como un producto.

LO QUE DEBE FUNCIONAR EN LA SESIÓN 13

PIEZA	ESTADO ESPERADO
Contratos	Desplegados y verificados en Sepolia, con cobertura ≥ 80 %.
Interfaz	Conecta la billetera, lee al menos un dato y ejecuta al menos una transacción.
Estados	Los cuatro errores y los estados pendiente/minada, con mensajes claros.
Repositorio	Contratos, pruebas y frontend, con un README que diga qué NO está descentralizado.

QUÉ DICE EL PLAN

El Avance 1 (S13) exige contratos desplegados y verificados, pruebas con 80 % de cobertura y repositorio documentado. La interfaz integrada se evalúa en el Avance 2 (S16) y en la rúbrica final («Integración e interfaz», 10 %: estados de transacción manejados y UX comprensible). Empezar hoy es llegar holgados.



LA FRASE DE INTEGRACIÓN

«Nuestra interfaz LEE ____ con una llamada view, y ESCRIBE ____ con una transacción que firma ____.»

- Si no la pueden completar en una línea, la interfaz aún no está pensada.
- Una función de punta a punta antes que diez a medias (corte vertical, como el proyecto del docente).
- Para el proyecto pueden usar React + wagmi; lo que se evalúa son los estados, no el framework.
- Reutilicen el manejo de errores del laboratorio: la tabla A.15 vale igual en React.

Verificación del bloque C: cada equipo lee su frase en voz alta. El docente pregunta «¿y si la transacción se revierte?».



LA PREGUNTA DE LA SEMANA

Abran una dApp conocida y, a mitad de uso, cambien de red en MetaMask o desconecten internet. ¿Qué deja de funcionar y qué mensaje les muestra? ¿Maneja los cuatro estados?

PARA LA PRÓXIMA SESIÓN

- Entregar la evidencia del laboratorio 12 (seis capturas, repo y reflexión).
- Avance 1 para la Sesión 13: contratos verificados + pruebas $\geq 80\%$ + README.
- Escribir la frase de integración del proyecto en el README.
- Leer sobre EIP-712 y «Sign-In with Ethereum» para la Sesión 13.



LO QUE SE LLEVAN DE HOY

await transfer() no dice «hecho», dice «enviado». Esa diferencia es la UX de Web3.

Ya hay una interfaz que lee y escribe en la cadena sin mentirle al usuario. La próxima sesión la completa: indexación, nombres legibles, firmas sin gas. Y se entrega el Avance 1.